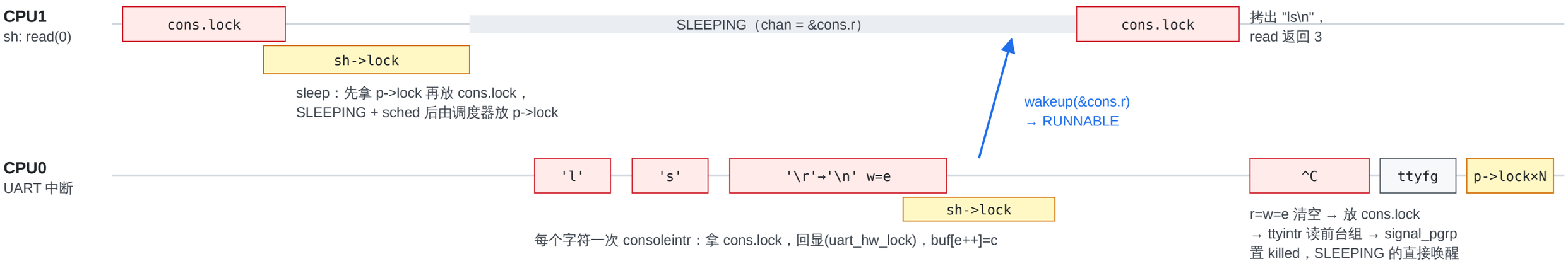
(a) 读一行 + ^C：cons.lock 与 p->lock 交接，保证不丢唤醒

时间从左到右。红底 = 持有 spinlock（同时本核关中断）。圆圈序号对应下方说明。



1. 为什么 sleep 不会丢唤醒：CPU0 的 wakeup 必须拿 sh->lock，而 CPU1 在放 cons.lock 之前就已持有 sh->lock，直到 sched() 切走后才放。
   所以“检查 r==w”和“变成 SLEEPING”对 consoleintr 来说是原子的（xv6 经典两把锁交接）。
2. 为什么 cons.lock 必须是 spinlock：生产者 consoleintr 在中断里跑，不能睡；acquire 里的 push_off 关本核中断，防止本核进程持锁时被 UART 中断再拿同一把锁自锁。
3. USB 键盘的 usbkbd_rx_work 在工作队列（进程上下文）调 consoleintr，同样被 cons.lock 串行化，和 UART 中断可在两个核上并发。

(b) 潜在死锁：PTY 回显写满 output 队列时，sshd 自己等自己

sshd 是单线程 epoll 循环（user/sshd.c relay_shell），output 队列唯一的读者就是 sshd 自己。



为什么会这样

- 回显是“同步、可阻塞”的：ptyecho 和普通 slave 输出共用 512 B 的 output 队列，ptyqput 满了就 sleep，而 master 侧写者恰好是唯一能消费 output 的人。
- epoll 也帮不上：ptypoll 对 master 的 EPOLLOUT 只看 input 队列是否有空位，不看回显要用的 output 队列。
- 只有 kill（p->killed）才能把 sshd 从 ptyqput 里拉出来；ssh 客户端那边表现为整个会话卡死。

Linux 的做法

- n_tty 把回显放进独立的 echo_buf，由 process_echoes() 在 output_lock 下按 tty_write_room() 尽力写出，写不下就留到下次，从不阻塞输入方。
- pty master 的 write 只把字节塞进对端 flip buffer（只受 input 侧空间限制），回显由 flush_to_ldisc 工作队列异步完成，写者不会等回显。
- 修复建议：ptyecho 改为非阻塞（output 满就丢回显，或单独小环），或在 ptyqput 中对 master→echo 情况直接返回；同时让 master 的 EPOLLOUT 也考虑 output 余量。